

## Ejercicio de refuerzo - Semana 1

Procesamiento reproducible de telemetría de servidores de IA

| Modalidad  | Tiempo sugerido | Entrega                       |
|------------|-----------------|-------------------------------|
| Individual | 2-3 horas       | Scripts, Parquet y reporte.md |

**Versión adaptada para el solver (Taller 03 v2, MMIA 6013).** El contenido técnico, los datos, las constantes y los resultados esperados son los del enunciado original del curso. Solo cambia la logística de entrega: no se usa Git, uv, pytest ni D2L; en lugar de completar un proyecto inicial, se entregan scripts de Python, el archivo Parquet y un *reporte.md*. Los contratos que verificaban las pruebas públicas se escriben aquí como requisitos.

### Objetivo

Integrar los contenidos de la primera semana en un único pipeline: representar datos con una abstracción simple, procesarlos con NumPy, analizarlos con Pandas y persistir el resultado en Parquet verificando el contrato de los datos.

### Escenario

El archivo `data/server_measurements.csv` contiene mediciones de seis servidores que ejecutan cargas de IA. Cada fila corresponde a una medición en un instante determinado. El archivo se entrega junto a este enunciado; no se descarga nada.

| Columna         | Tipo       | Descripción                                                  |
|-----------------|------------|--------------------------------------------------------------|
| server          | texto      | identificador del servidor (AI-SRV-01 ... AI-SRV-06)         |
| timestamp       | fecha-hora | instante de la medición (leer con <code>parse_dates</code> ) |
| gpu_utilization | float      | utilización de GPU, %                                        |
| cpu_utilization | float      | utilización de CPU, %                                        |
| memory_gb       | float      | memoria en uso, GB                                           |
| power_w         | float      | potencia, W                                                  |
| temperature_c   | float      | temperatura, °C                                              |

### Reglas

- Usar como características exactamente `gpu_utilization`, `cpu_utilization` y `memory_gb`, en ese orden.
- Las operaciones de las Tareas 2 y 3 deben ser vectorizadas: **no recorrer las observaciones con un for**.
- **No modificar el DataFrame original**: toda columna nueva va en una copia.
- No modificar el CSV de entrada.

### Tarea 1 - Representar las mediciones

Implementar una clase `ServerMeasurements` que almacene una matriz NumPy bidimensional (`values`) y una tupla con los nombres de las características (`feature_names`). La clase debe lanzar `ValueError` si la matriz no es bidimensional, si el número de columnas no coincide con la cantidad de nombres o si los datos no son numéricos. Implementar también `__repr__`, que muestre la forma y los nombres.

Implementar `build_measurements(df)`: selecciona las tres características, las convierte a una matriz NumPy de tipo flotante y retorna un objeto `ServerMeasurements`.

Resultado esperado:

```
ServerMeasurements(shape=(300, 3), features=('gpu_utilization',  
'cpu_utilization', 'memory_gb'))
```

### Tarea 2 - Normalizar y calcular un indicador de carga

Calcular media y desviación estándar por columna. Normalizar las tres características mediante Z-score. Si una desviación estándar es cero, evitar la división por cero. Después calcular `load_score` usando los pesos (0.50, 0.30, 0.20) y operaciones vectorizadas.

Resultado esperado:

```
means: [57.99313333 50.54163333 29.45533333]
zscores.shape -> (300, 3)
load_score.shape -> (300,)
```

Cada columna de `zscores` debe tener media 0 y desviación 1 (tolerancia 1e-10), y `load_score` debe ser igual a `zscores @ [0.50, 0.30, 0.20]`.

### Tarea 3 - Identificar observaciones para revisión

Crear un nuevo DataFrame a partir del original y agregar las columnas `load_score` y `requires_review` (tipo bool). Una fila requiere revisión si `load_score > 1.5` o `temperature_c > 80`.

Resultado esperado (una fila):

| server       | timestamp           | load_score | temperature_c | requires_review |
|--------------|---------------------|------------|---------------|-----------------|
| 23 AI-SRV-01 | 2026-08-24 09:55:00 | 3.136218   | 66.97         | True            |

### Tarea 4 - Construir un resumen por servidor

Agrupar por `server` y construir una tabla con una fila por servidor y exactamente estas columnas, en este orden: `server`, `observations`, `mean_power_w`, `max_temperature_c`, `mean_load`, `review_count`. `review_count` cuenta las observaciones del servidor con `requires_review=True`.

Resultado esperado (una fila):

| server    | observations | mean_power_w | max_temperature_c | mean_load | review_count |
|-----------|--------------|--------------|-------------------|-----------|--------------|
| AI-SRV-01 | 50           | 363.4598     | 86.50             | -0.614527 | 2            |

### Tarea 5 - Guardar y verificar el resultado

Guardar el DataFrame analizado de la Tarea 3 en `output/server_analysis.parquet` mediante PyArrow, sin el índice. Volver a leer el archivo y verificar, como mínimo: filas antes == filas después, columnas antes == columnas después, esquema antes == esquema después y valores iguales.

Resultado esperado:

```
Round trip verificado: output/server_analysis.parquet
Filas: True
Columnas: True
Esquema: True
Valores: True
```

### Entregable

- El código fuente en Python que produce todos los resultados.
- `output/server_analysis.parquet`.
- `reporte.md` con una sección por tarea, en orden (Tarea 1 a Tarea 5), que cite los resultados obtenidos por el código: el repr, las medias, la fila 23, la tabla resumen completa (seis servidores) y las cuatro verificaciones del round trip. Máximo 800 palabras.
- Toda cifra del reporte debe provenir de una ejecución del código.